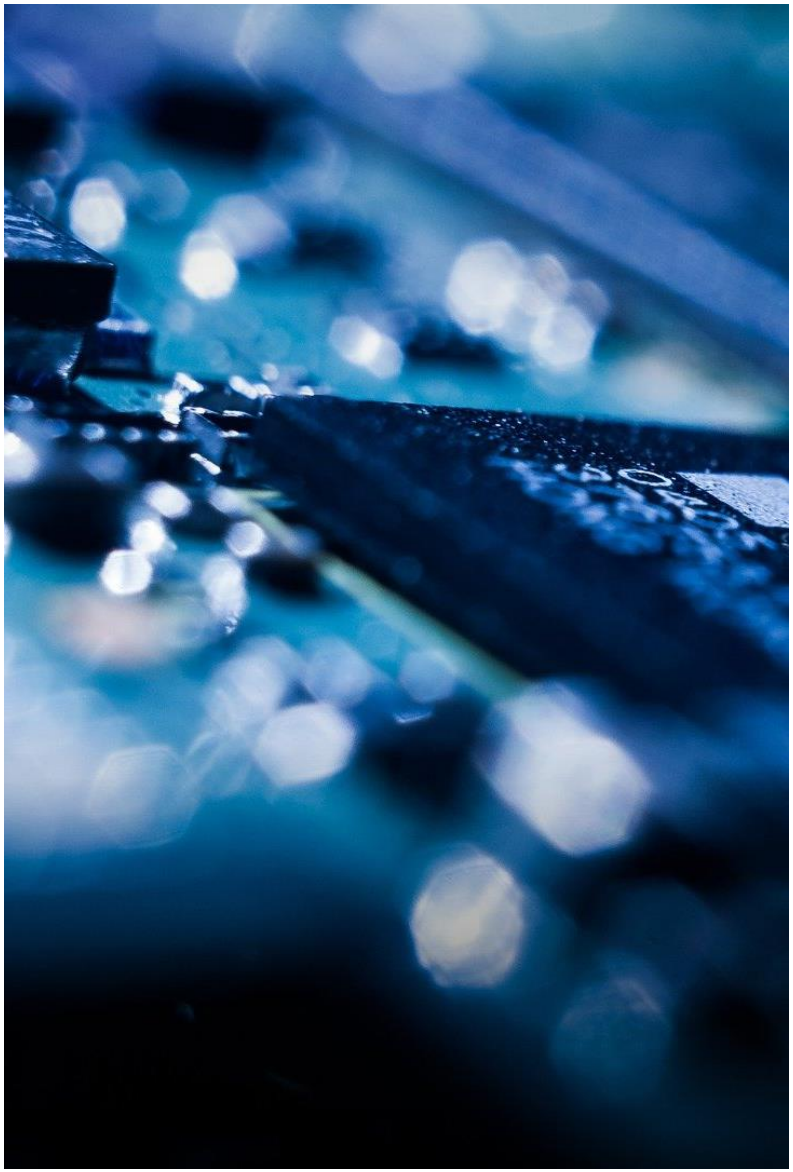
Programmierung eingebetteter Systeme

Unterprogramme



Prof. Dr. Henning Trsek
Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de

Überblick – Unterprogramme

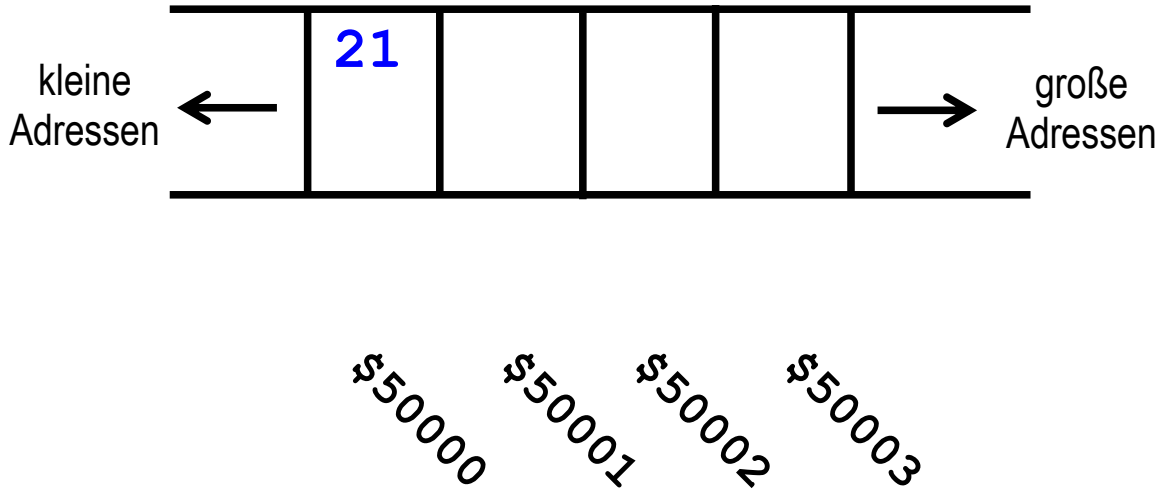
- Exkurs: Daten im Speicher
- Unterprogramme allgemein
- Beispiele für Unterprogramme
- Benutzung des Stacks

Datenablage im Speicher (Big und Little Endian)

Big Endian
(Beispiel: Datenregister -> Speicheradr.$50000)

Datenregister: **$87654321**

```
MOVE.B r0, $50000
```

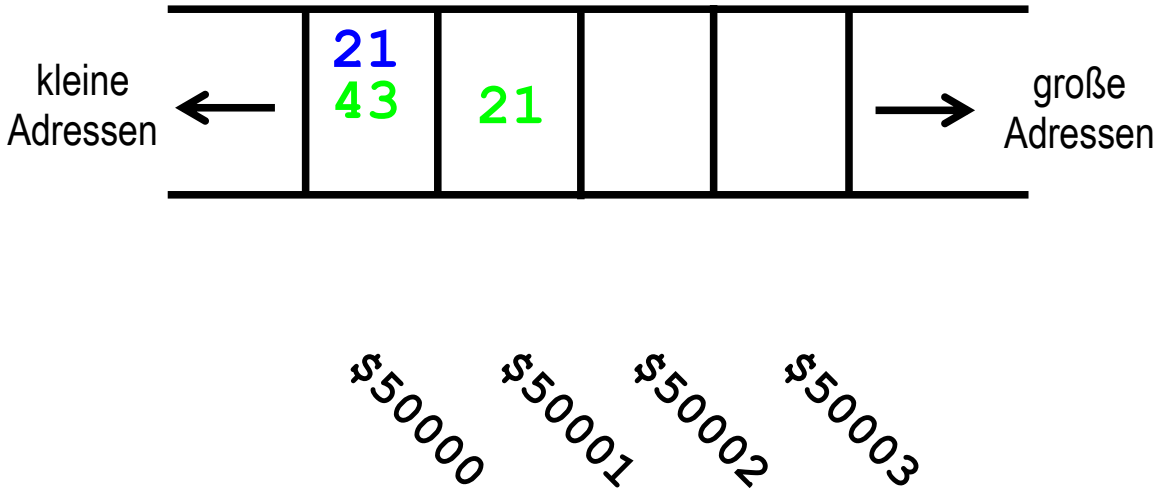


Datenablage im Speicher (Big und Little Endian)

Big Endian
(Beispiel: Datenregister -> Speicheradr.$50000)

Datenregister: **$87654321**

```
MOVE.B r0, $50000

MOVE.W r0, $50000
```

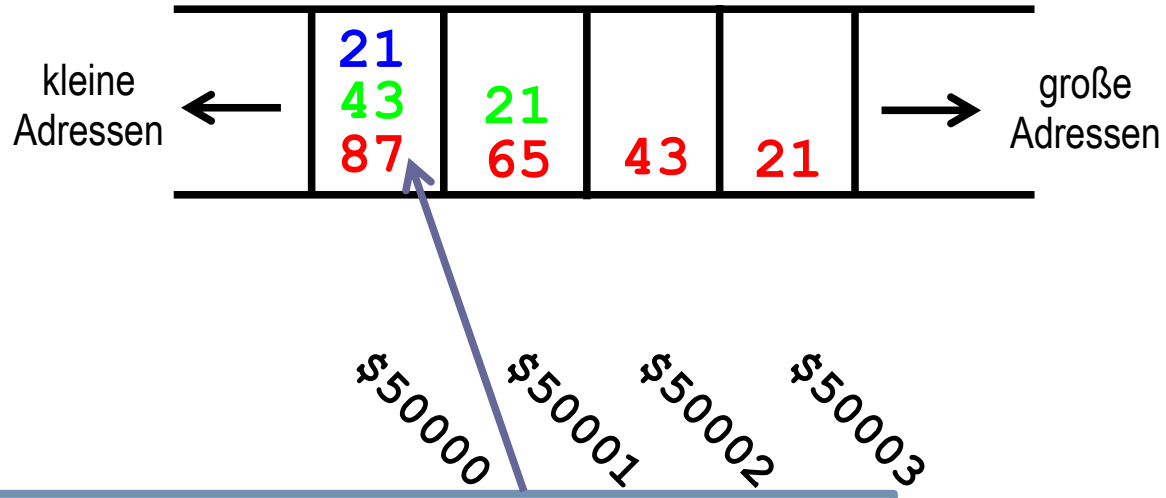


Datenablage im Speicher (Big und Little Endian)

Big Endian
(Beispiel: Datenregister -> Speicheradr.$50000)

Datenregister: $87654321

```
MOVE.B r0, $50000

MOVE.W r0, $50000

MOVE.L r0, $50000
```



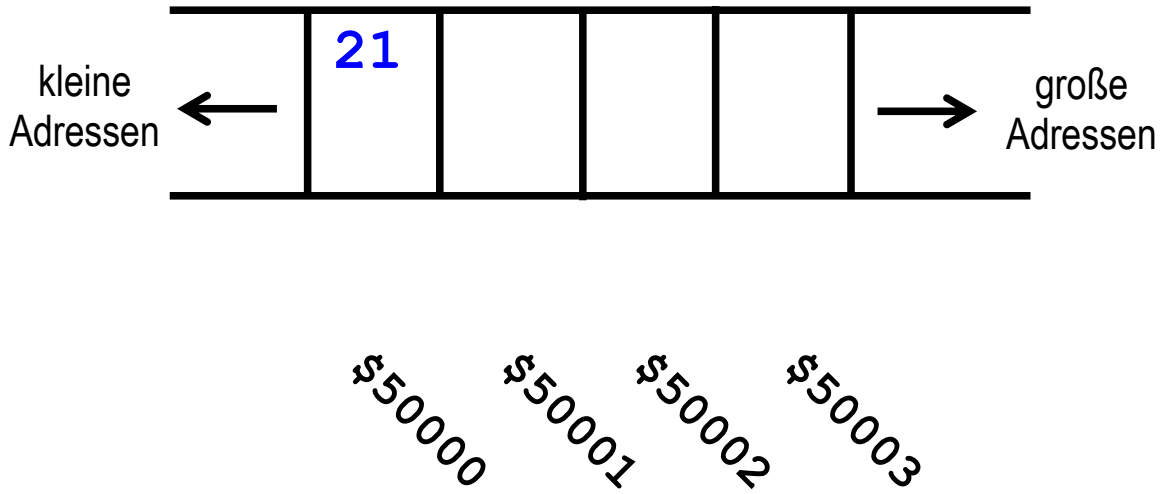
Big Endian
Das Byte mit der höchsten Wertigkeit steht auf der niedrigsten Adresse

Datenablage im Speicher (Big und Little Endian)

Little Endian
(Beispiel: Datenregister -> Speicheradr.$50000)

Datenregister: **$87654321**

```
MOVE.B r0, $50000
```

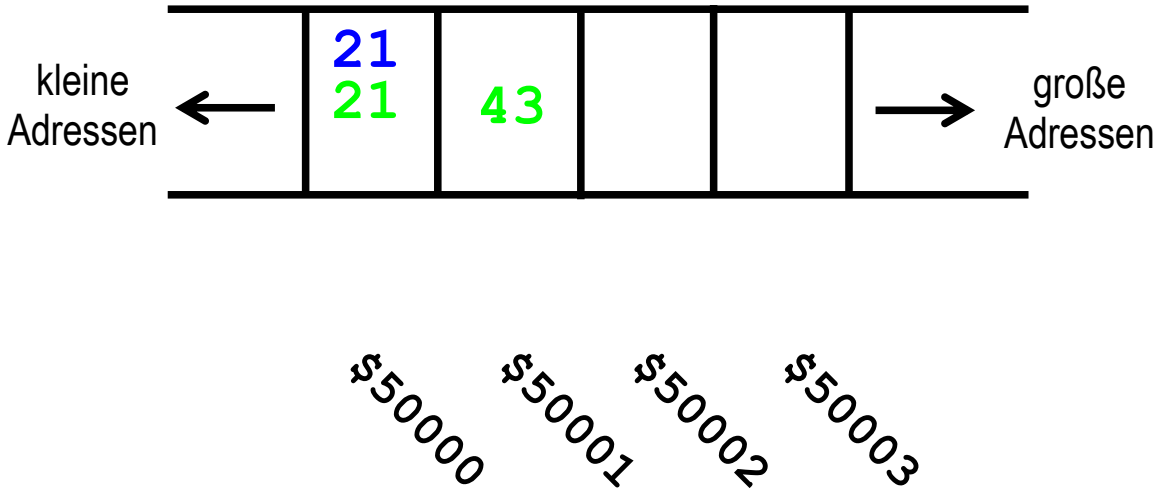


Datenablage im Speicher (Big und Little Endian)

Little Endian
(Beispiel: Datenregister -> Speicheradr.$50000)

Datenregister: **$87654321**

```
MOVE.B r0, $50000

MOVE.W r0, $50000
```

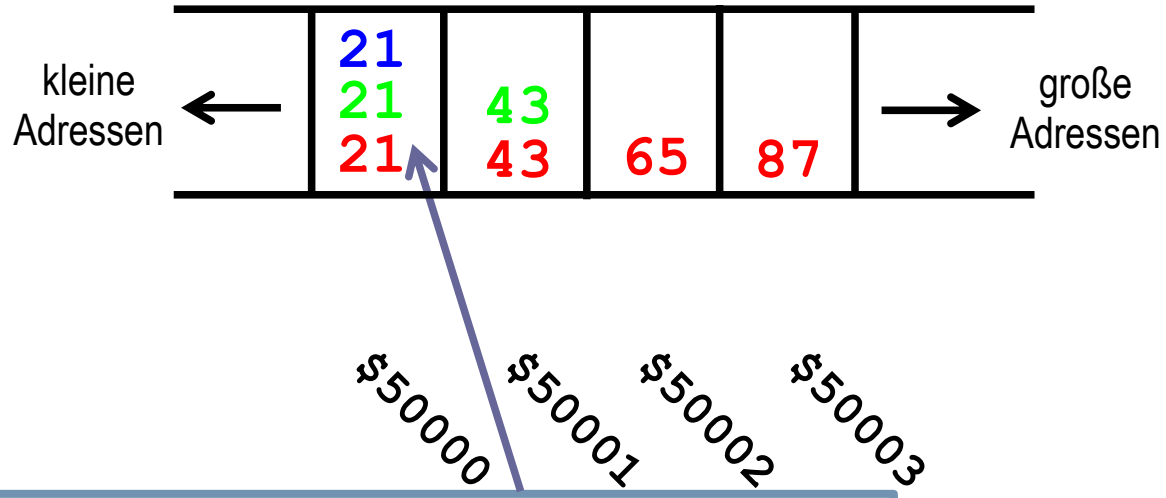


Datenablage im Speicher (Big und Little Endian)

Little Endian
(Beispiel: Datenregister -> Speicheradr. $50000)

Datenregister: $87654321

```
MOVE.B r0, $50000

MOVE.W r0, $50000

MOVE.L r0, $50000
```

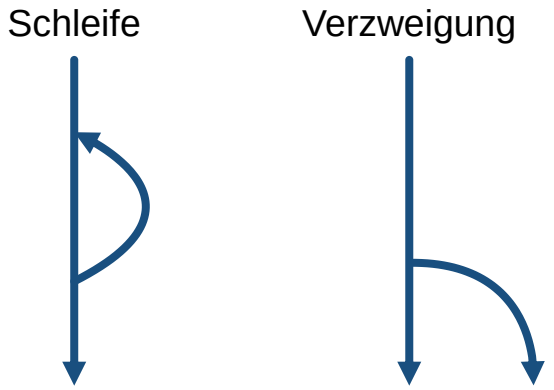


Little Endian
Das Byte mit der niedrigsten Wertigkeit steht auf der niedrigsten Adresse

Unterprogramme

Möglichkeiten bisher:

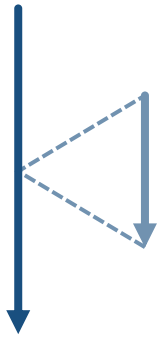
- sequentiellen Programmfluss verlassen mit `bra`, `beq` usw.



Rückkehr zur "Absprungstelle"?

Möglichkeiten mit Unterprogrammen:

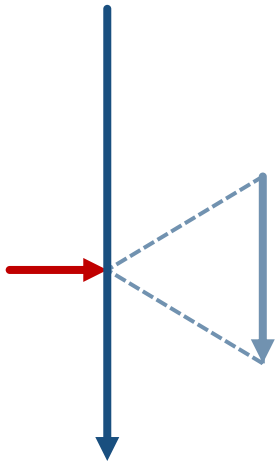
- Programmteile mehrfach nutzen
- Strukturierung des Programms



Rückkehr zum Hauptprogramm?

Unterprogramme

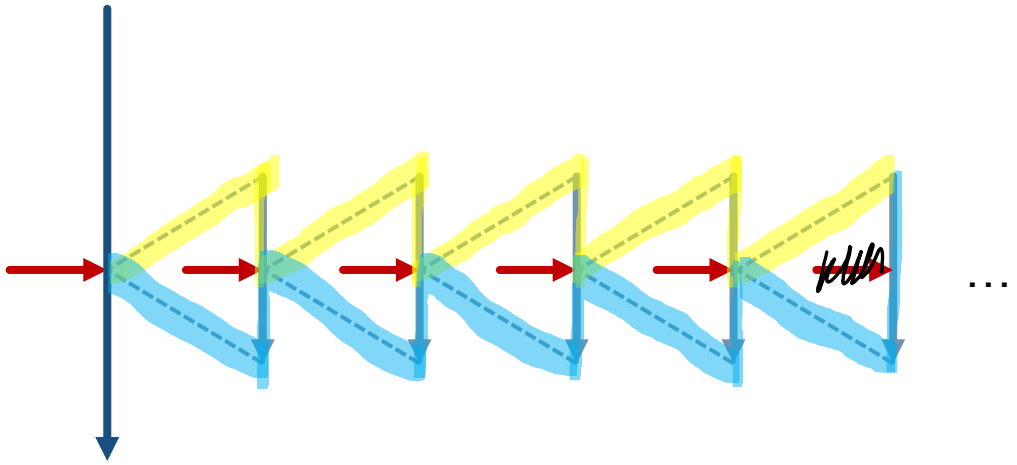
Rücksprung zum Hauptprogramm



Rücksprungadresse speichern

Unterprogramme

Rücksprung zum Hauptprogramm



Allgemeiner Fall: mehrere Rücksprungadressen speichern

Unterprogramme

Rücksprung zum Hauptprogramm

Speicherbereich erforderlich, in dem die Rücksprungadressen abgelegt sind

Besonderheit: Es wird zu einem Zeitpunkt nur die Rücksprungadresse gebraucht, die als letzte in den Speicherbereich geschrieben wurde.

Stack: Speicherbereich, in dem die Rücksprungadressen abgelegt sind.

last in – first out

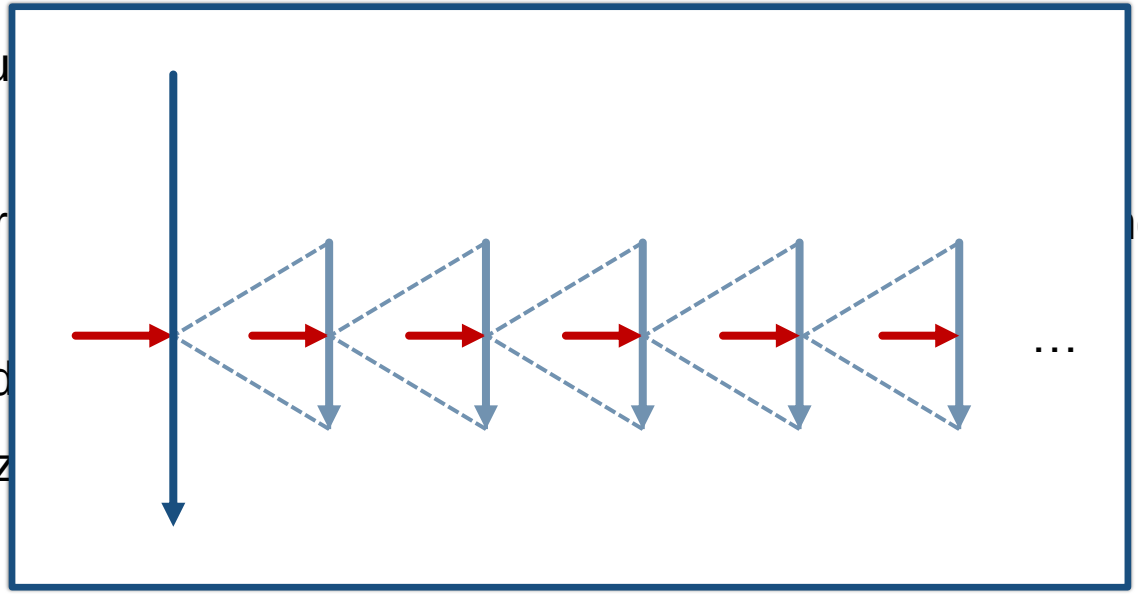
Unterprogramme

Rücksprung zum Hau

Speicherbereich erfor ıd

Besonderheit: Es wird

gebraucht, die als letz



Stack: Speicherbereich, in dem die Rücksprungadressen abgelegt sind.

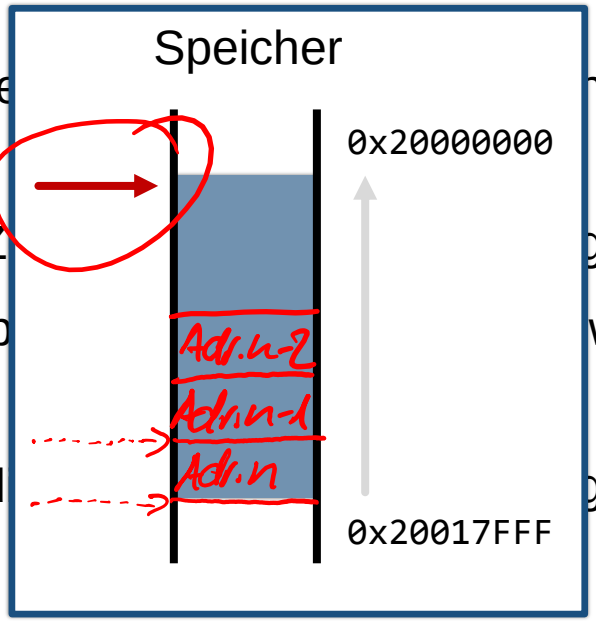
last in – first out

Unterprogramme

Rücksprung zum Hauptprogramm

Speicherbereich erforderlich, in de n abgelegt sind

Besonderheit: Es wird zu einem Z gadresse gebraucht, die als letzte in den Sp wurde.



Stack: Speicherbereich, in dem d gelegt sind.

last in – first out

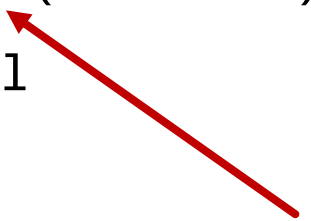
Stackpointer sp: Register, das auf den aktuellen Stackeintrag zeigt.

Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}  label
```



schreibt die Adresse der nächsten Anweisung in LR (Link Register, R14)

Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}  label
```

BX

Branch Indirect (Register)

```
bx{cond}  Rm
```

```
bx  lr
```

Branch to Link Register

Unterprogrammsprung – Befehle

BL

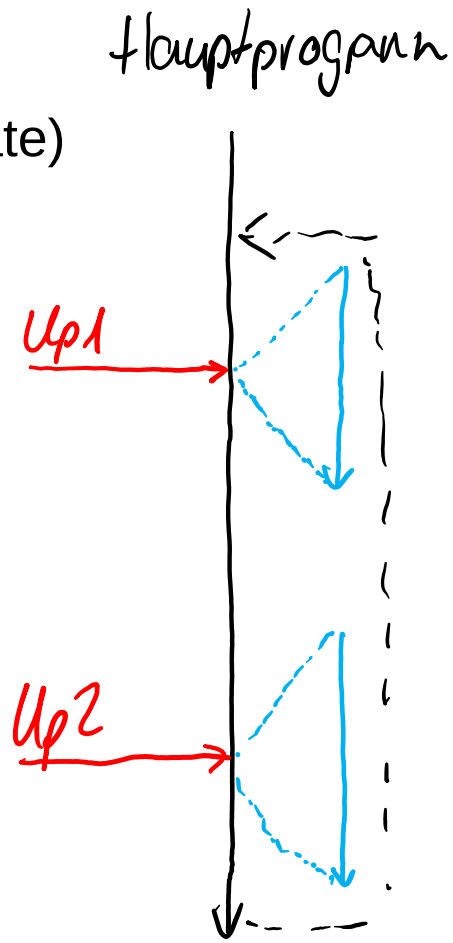
Branch with Link (Immediate)

```
bl{cond}   label
```

BX

Branch Indirect (Register)

```
bx{cond}   Rm
bx   lr
```




```
loop: bl   up1
      bl   up2
      ...
@-----Unterprogramm 1-----
up1:  ldr ...
      ...
      bx   lr
      ...
@-----Unterprogramm 2-----
up2:  str ...
      ...
      bx   lr
```

Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

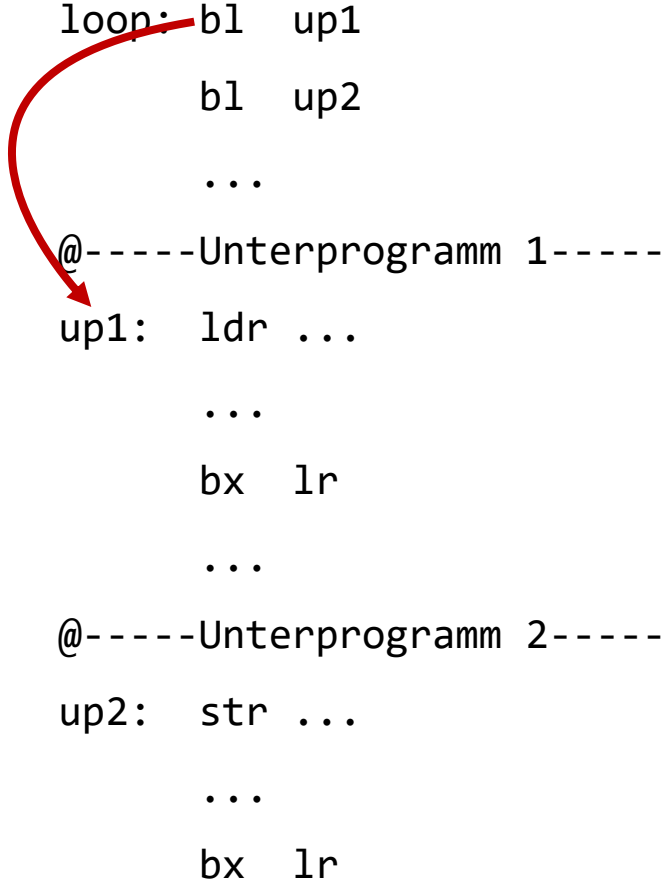
```
bl{cond}   label
```

BX

Branch Indirect (Register)

```
bx{cond}   Rm
```

```
bx   lr
```



Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}   label
```

BX

Branch Indirect (Register)

```
bx{cond}   Rm
```

```
bx   lr
```

```
loop: bl   up1
      bl   up2
      ...
@-----Unterprogramm 1-----
up1:  ldr ...
      ...
      bx   lr
      ...
@-----Unterprogramm 2-----
up2:  str ...
      ...
      bx   lr
```

Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}   label
```

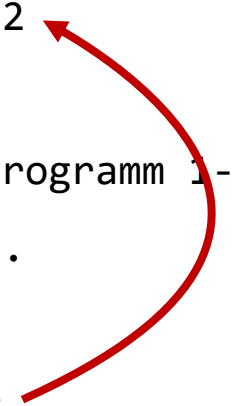
BX

Branch Indirect (Register)

```
bx{cond}   Rm
```

```
bx   lr
```

```
loop: bl   up1
      bl   up2
      ...
@-----Unterprogramm 1-----
up1:  ldr ...
      ...
      bx   lr
      ...
@-----Unterprogramm 2-----
up2:  str ...
      ...
      bx   lr
```



Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

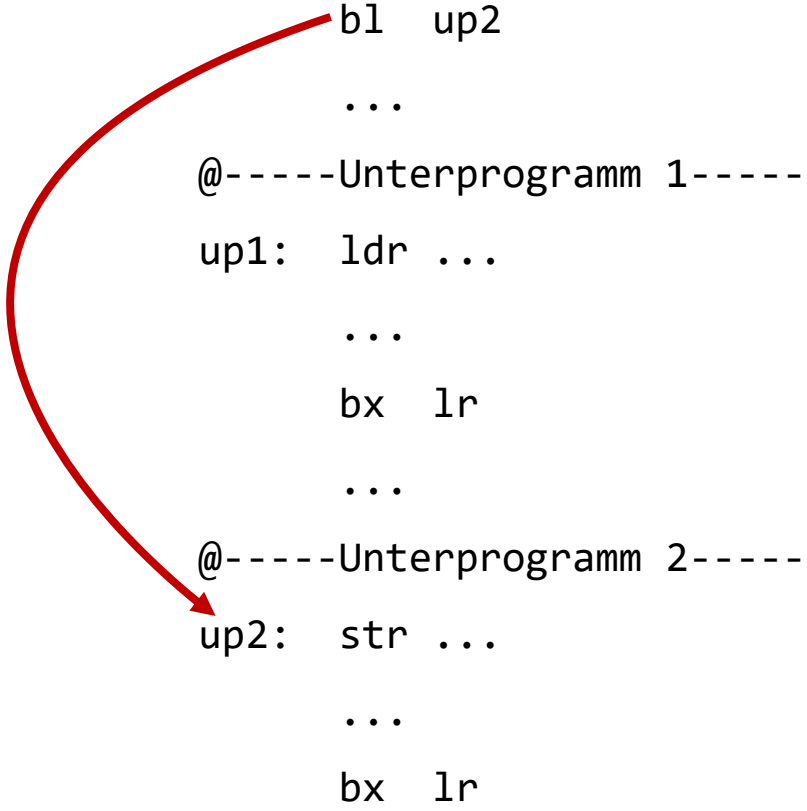
```
bl{cond}   label
```

BX

Branch Indirect (Register)

```
bx{cond}   Rm
```

```
bx   lr
```



Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}   label
```

BX

Branch Indirect (Register)

```
bx{cond}   Rm
```

```
bx   lr
```

```
loop: bl   up1
      bl   up2
      ...
@-----Unterprogramm 1-----
up1:  ldr ...
      ...
      bx   lr
      ...
@-----Unterprogramm 2-----
up2:  str ...
      ...
      bx   lr
```

Unterprogrammsprung – Befehle

BL

Branch with Link (Immediate)

```
bl{cond}   label
```

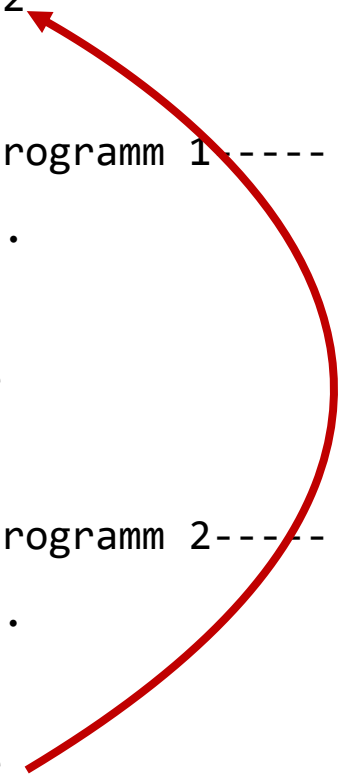
BX

Branch Indirect (Register)

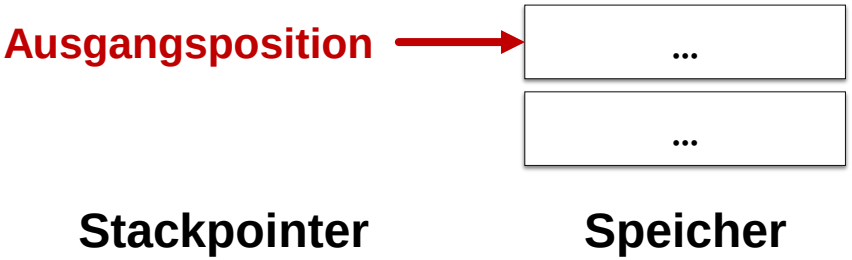
```
bx{cond}   Rm
```

```
bx   lr
```

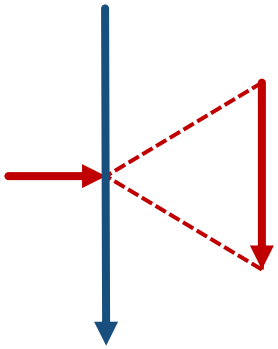
```
loop: bl   up1
      bl   up2
      ...
@-----Unterprogramm 1-----
up1:  ldr ...
      ...
      bx   lr
      ...
@-----Unterprogramm 2-----
up2:  str ...
      ...
      bx   lr
```

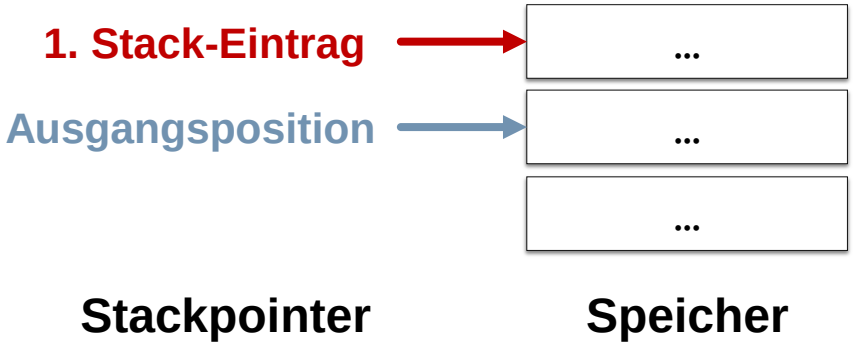


Unterprogrammsprung – Allgemein

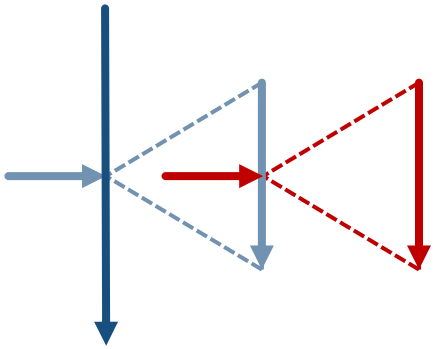


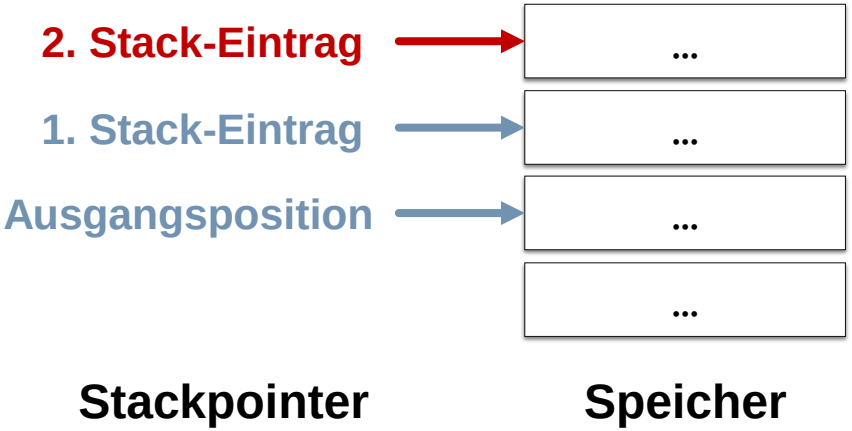
Unterprogrammsprung – Allgemein



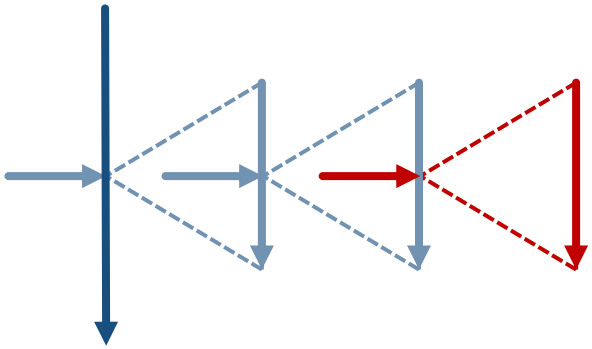


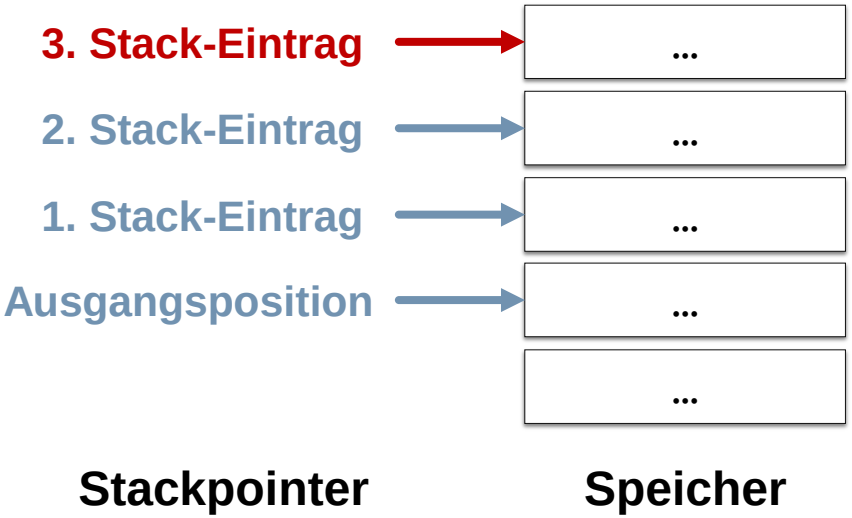
Unterprogrammsprung – Allgemein



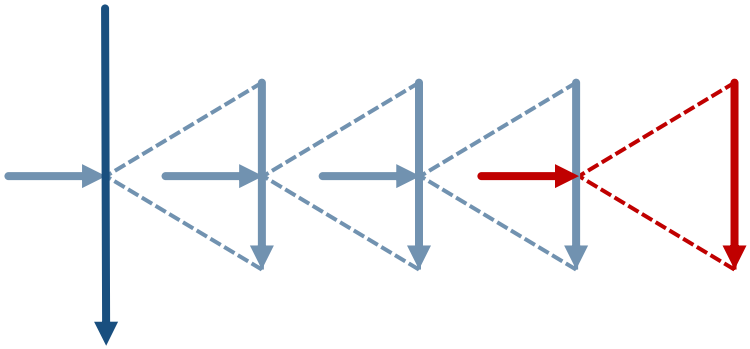


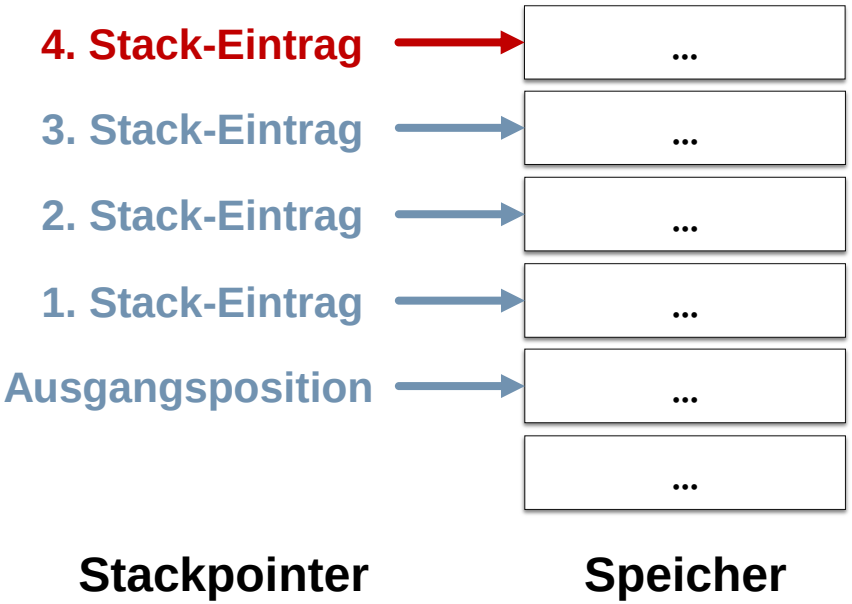
Unterprogrammsprung – Allgemein



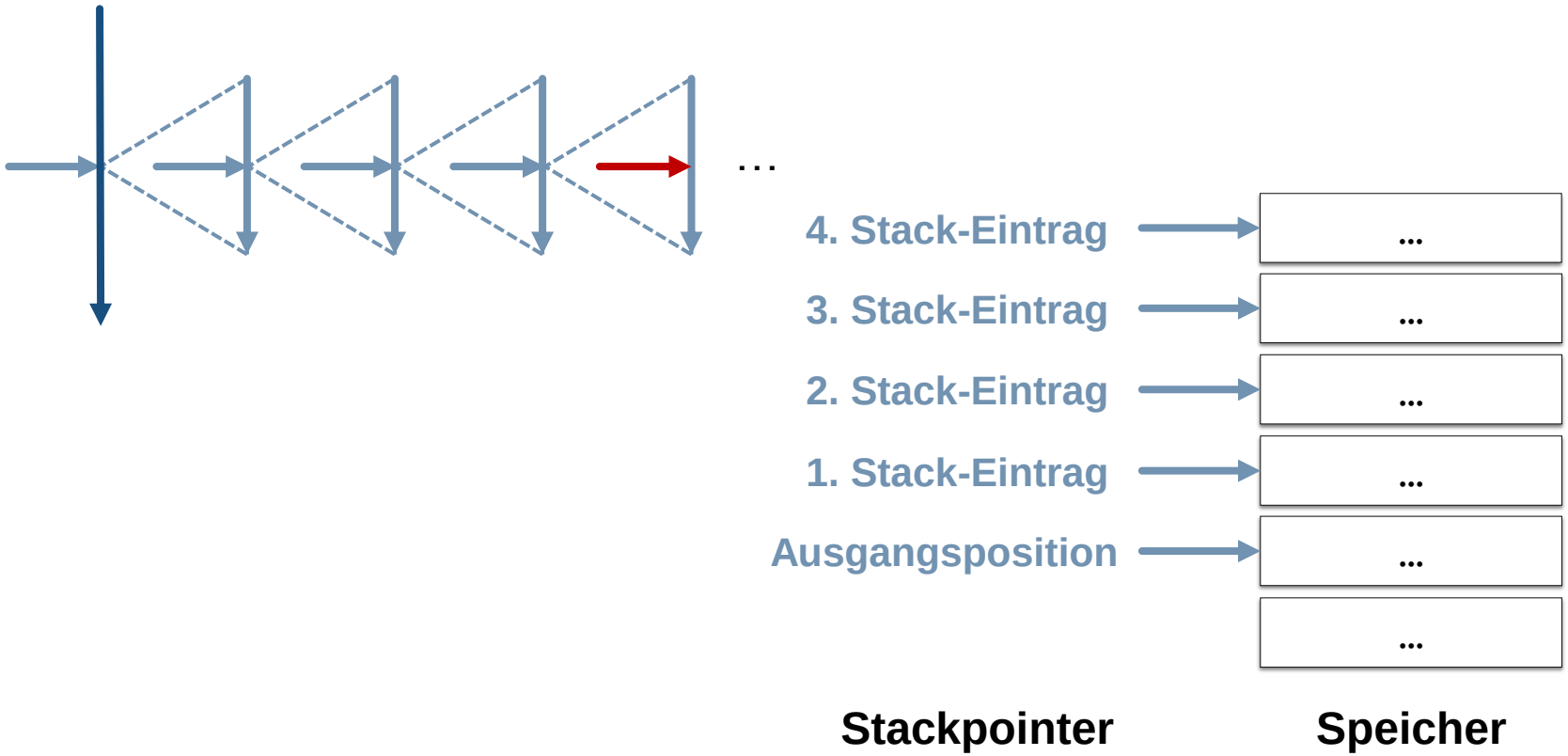


Unterprogrammsprung – Allgemein





Unterprogrammsprung – Allgemein



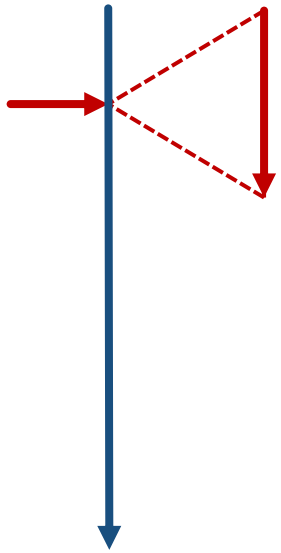
Beispiele: Unterprogramme

Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
         bl   prog2

ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```

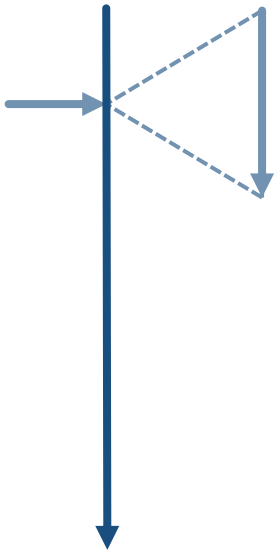
Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
         bl   prog2

ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



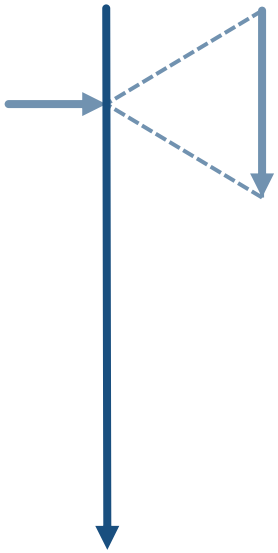
Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
lr  ->   bl   prog2

ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



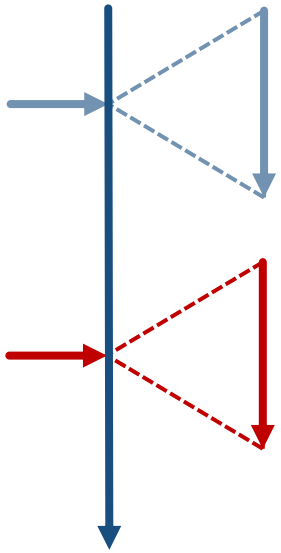
Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
lr  ->   bl   prog2

ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



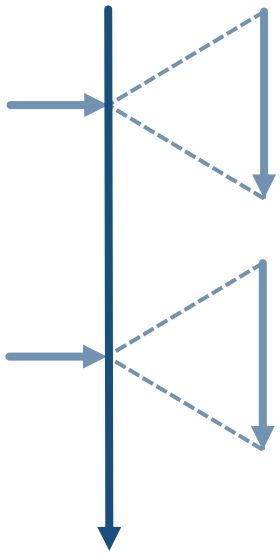
Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
         bl   prog2

ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



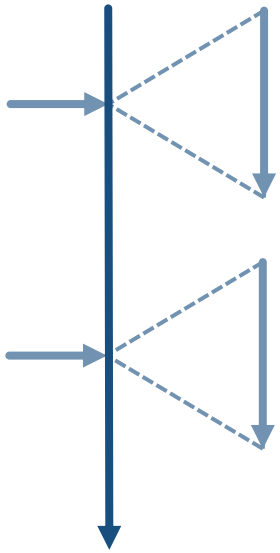
Unterprogrammsprung – Beispiel 1

```
prog:    mov  r1, #1
         bl   prog1
         bl   prog2

lr →
ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



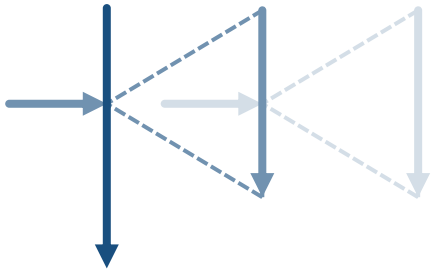
Unterprogrammsprung – Beispiel 1

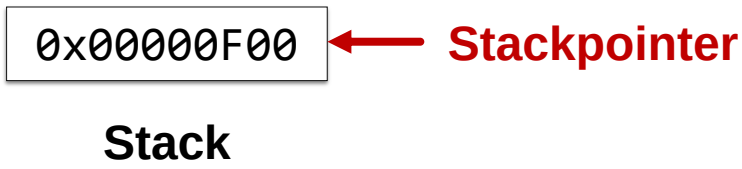
```
prog:    mov  r1, #1
         bl   prog1
         bl   prog2

lr →     ende:    nop
         b    ende

prog1:   mov  r1, #1
         bx   lr

prog2:   add  r1, #1
         bx   lr
```



Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

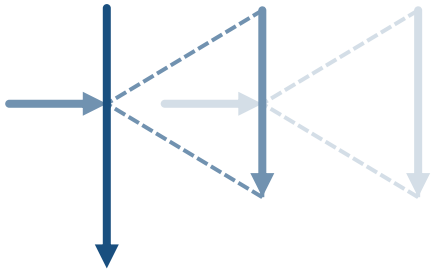




Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

lr



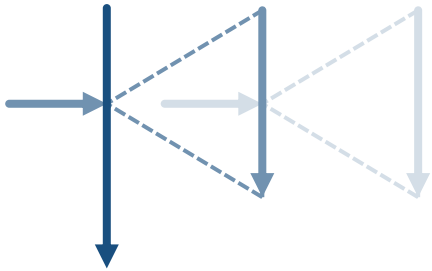
0x00000F00 ← Stackpointer

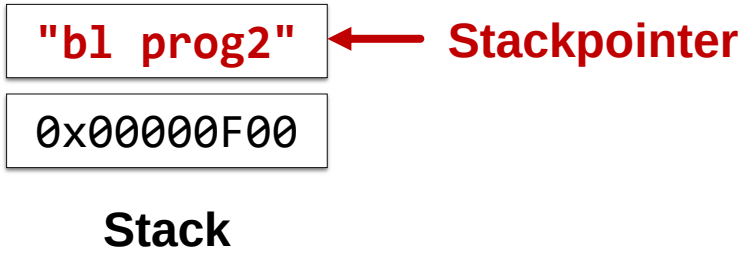
Stack

Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

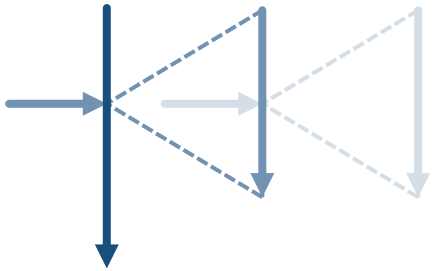
lr

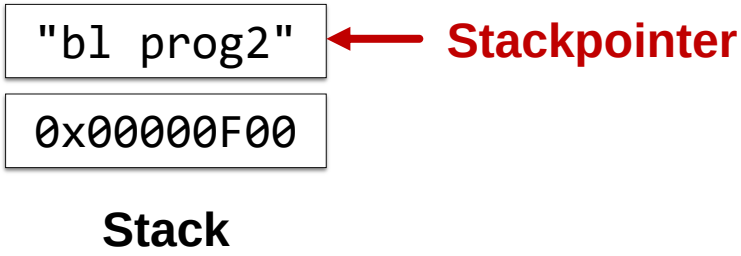




Unterprogrammsprung – Beispiel 2

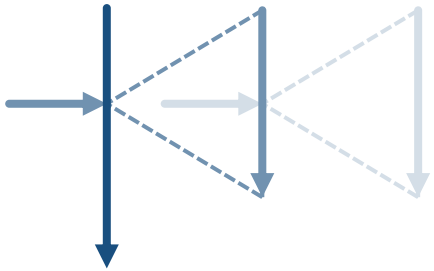
```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

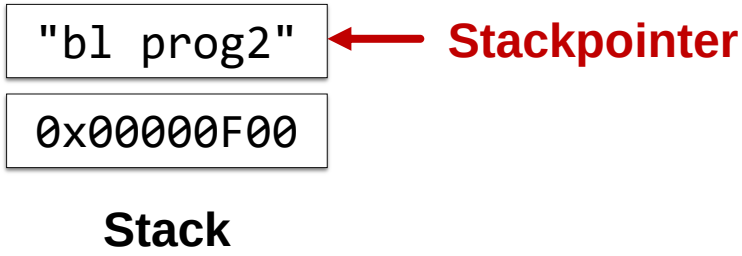




Unterprogrammsprung – Beispiel 2

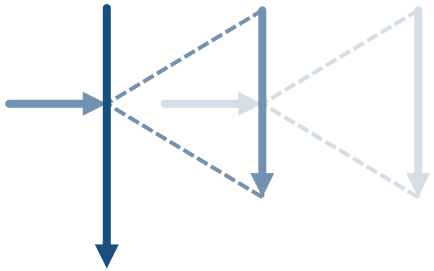
```
stk:     bl   prog1
         bl   prog2
ende:    nop
         b    ende
prog1:   push {lr}
         mov  r1, #1
         bl   prog3
lr  ->   pop  {pc}
prog2:   add  r1, #1
         bx   lr
prog3:   push {lr}
         add  r1, #1
         bl   prog4
         pop  {pc}
prog4:   add  r1, #1
         bx   lr
```

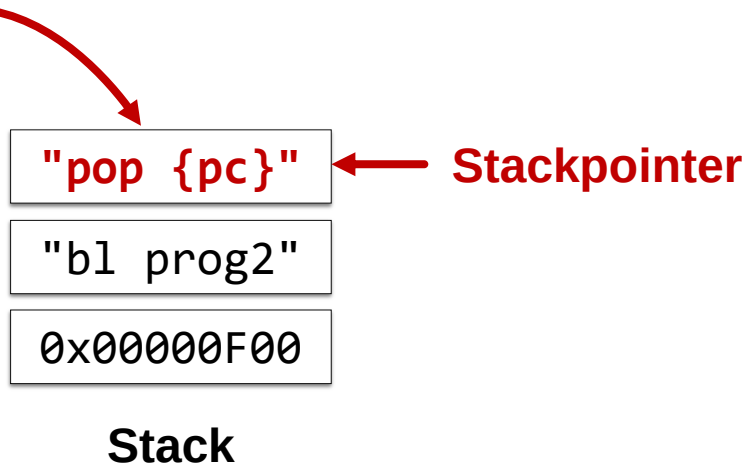




Unterprogrammsprung – Beispiel 2

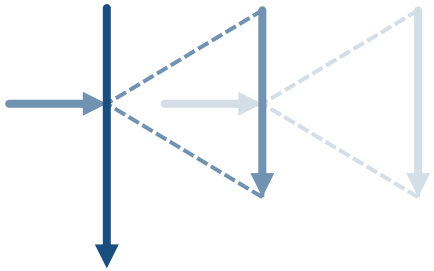
```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
lr ->   pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

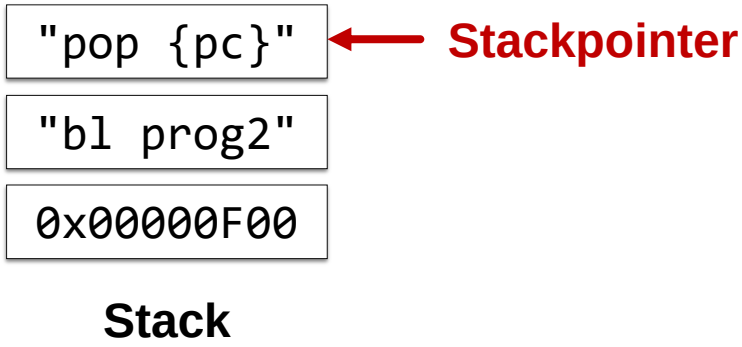




Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

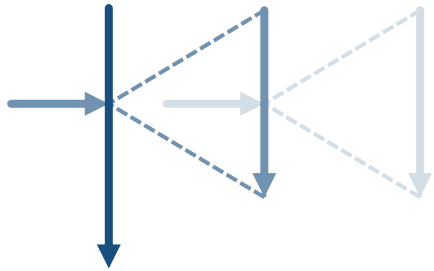


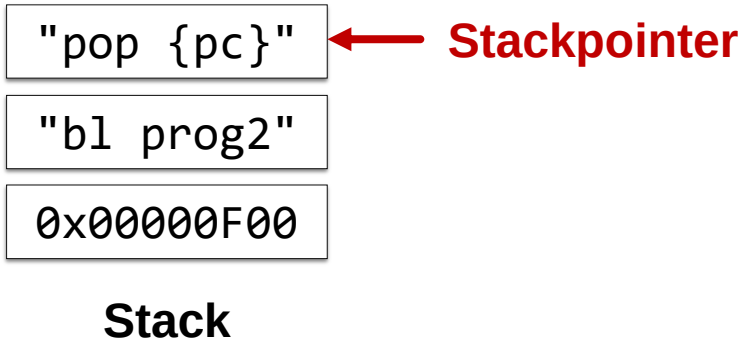


Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

lr

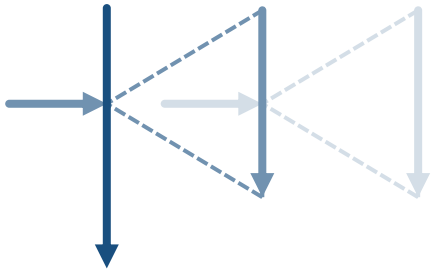


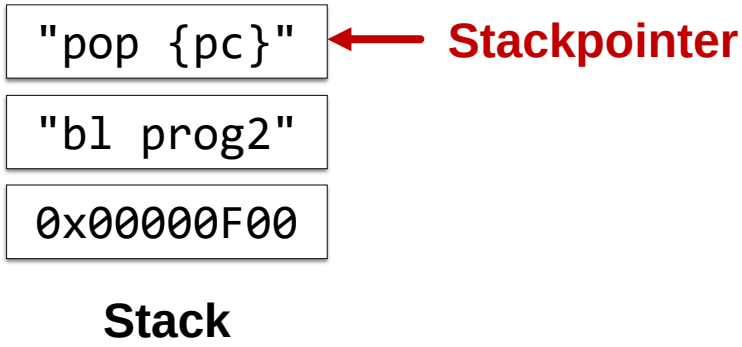


Unterprogrammsprung – Beispiel 2

```
stk:     bl   prog1
         bl   prog2
ende:    nop
         b    ende
prog1:   push {lr}
         mov  r1, #1
         bl   prog3
         pop  {pc}
prog2:   add  r1, #1
         bx   lr
prog3:   push {lr}
         add  r1, #1
         bl   prog4
         pop  {pc}
prog4:   add  r1, #1
         bx   lr
```

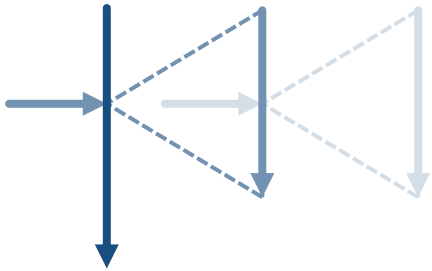
lr

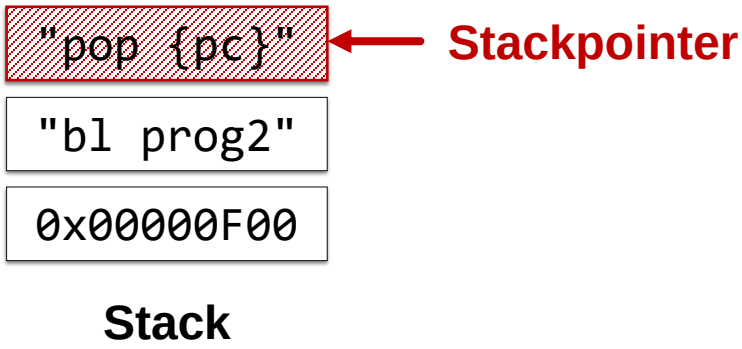




Unterprogrammsprung – Beispiel 2

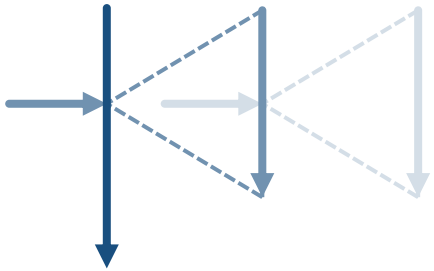
```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

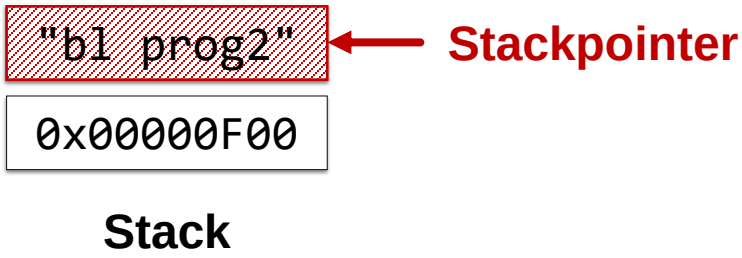




Unterprogrammsprung – Beispiel 2

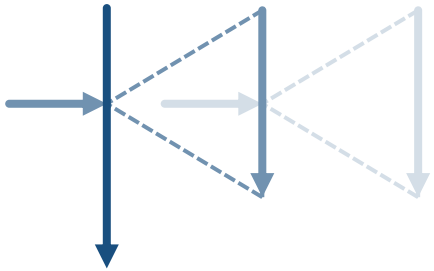
```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

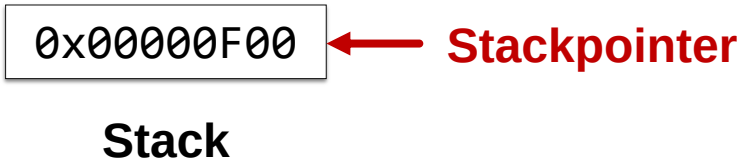




Unterprogrammsprung – Beispiel 2

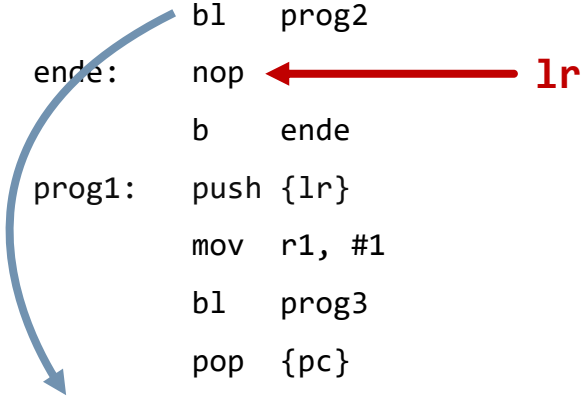
```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

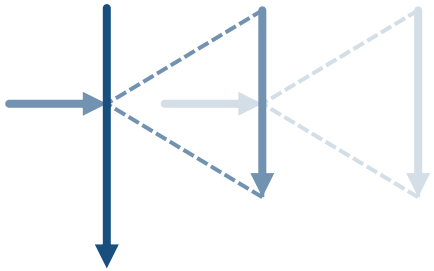


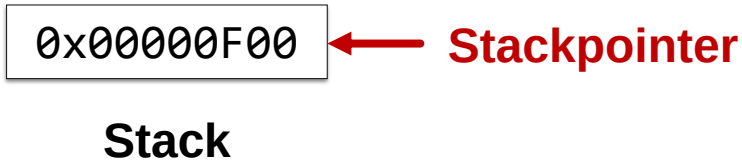


Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```



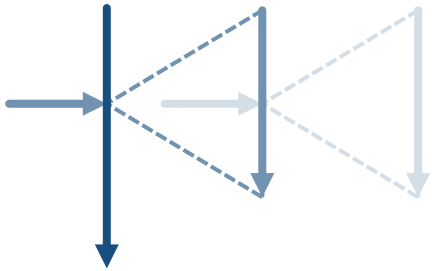




Unterprogrammsprung – Beispiel 2

```
stk:    bl   prog1
        bl   prog2
ende:   nop
        b    ende
prog1:  push {lr}
        mov  r1, #1
        bl   prog3
        pop  {pc}
prog2:  add  r1, #1
        bx   lr
prog3:  push {lr}
        add  r1, #1
        bl   prog4
        pop  {pc}
prog4:  add  r1, #1
        bx   lr
```

lr



0x00000F00 ← **Stackpointer**

Stack

Programmstruktur mit Unterprogrammen